

Azure Data Engg

Azure is one Cloud Platform to perform various Tasks/Projects by utilizing of its Services in various sectors. In Azure DE, we need to handle Bigdata i.e. where huge data (Peta/Exa/Zeta bytes) needs to be processed, there some solutions using, i.e. GCP-DE, Azure DE, AWS DE services in Cloud-wise and similarly on-premises Hadoop [HDFS(Storage) + Hive(Processing) + Functions {MapReduce(old)/Spark(New)}]

Azure-DE Services:

ADLS, ADF, Synapse, Databricks, Bigdata [PySpark + SQL]

Topics:

IR:

Azure IR: It provides server-less fully Azure Managed Compute services with public accessible end points and it is easily scalable according to workloads. We can check it in ADF --> Monitor --> IR

ADF --> Author --> Connections --> Integration Runtimes --> New --> Azure/Self/SSIS IRs

IR will be Mandatory used in Linked Service creation.

Self-Hosted IR: It is used connection b/w cloud & on-premises/Pvt Networked VMs. Currently support for Windows only
This Self-Hosted IR needs to be installed on on-prem/Azure VMs in Pvt VNets

ADF --> Author --> Connections --> Integration Runtimes --> New --> Self-Hosted --> Name --> create

Settings --> Express Setup/Manual Setup --> download IR (.msi) on Laptop, Next double click it & Install, Finish

Settings --> copy key1/key2 and paste into Register IR(self-host) window --> Register --. Finish

Next we can use above IR in Linked Connections b/w Azure and On-prem

Parameterization:

ADF --> Author --> Connections --> Linked Services --> New --> SQL server(on-prem) --> Name, Azure IR, ,
parameter(New --> dbName, serverName, UserName, password), Server Name(dynamic content, select serverName), DB
Name(dynamic content, select dbName), User Name(dynamic content, select userName), Password(dynamic content,
select password) --> create

Dataset --> parameters(New, tableName, dbName) --> connection [DB(dynamic content(select dbName),
(Table Edit(dbo.dynamic content(select tableName)) --> Finish

Pipeline --> parameters(sourceTableName_pip, DbName_pip, sinkTableName_pip), drag copy activity --> Source
(dynamic content select sourceTableName_pip, DbName_pip), Sink(dynamic content select sinkTableName_pip,
DbName_pip) --> Finish --> Publish all --> Trigger

Connector:

In ADF, which are using as Source/Sink, they are Connectors [ADLS, SQL-DB....etc] i.e. datastores

Activities:

Copy:

ADF --> Author --> pipeline --> drag copy --> configure source, sink with datasets, linked services, needed file formats
Source [ADLS, csv file] --> Sink [ADLS, csv file], Timeout, Retry, Retry interval, File path type, Mapping (schema)...etc
As per dataset Types (ADLS, SQL...etc), above configuring details will be changed.

Inside of Copy Activity, in Settings (we can configure compute capacity)

Specific File within ADLS: In this case, we need to select specific csv file in source dataset, remain same as above

Copy all content within ADLS: In this case, source dataset--> wildcard filepath(container/folder/*.csv), remain same as above

Copy Specific Multiple Files within ADLS: In this case, source dataset--> List of Files, path to file list(Browse txt file
consists of all file names to be copied), remain all same as above

Copy ADLS --> Azure SQL DB: In this case, source dataset--> select needed csv file, Sink Dataset --> config SQL DB details
For this case, we need to keep ready Azure SQL DB, if Table not prepared in DB, select Auto create Table option in sink,
Table(Edit) --> dbo.nn-table --> remain Trigger pipeline same as above

For SQL DB --> ADLS case, we need to kept ready SQL DB Table with data, sink(ADLS) remain same as above

Add Additional Columns: In this case, Source(ADLS, wildcard(*.csv)), sink(SQL DB, Auto create Table)

Source, Add columns, New --> (Filename, \$\$FILEPATH), Table(Edit) --> dbo.nn-table, remain debug pipeline same

Copy ADLS --> Azure SQL DB with Timestamp: In this case, source(ADLS), sink(SQL DB, Auto create Table)

Source, Add columns, New --> (DateProcessed, dynamic content, utcnow), Table(Edit) --> dbo.nn-table, remain debug
pipeline same as above

Copy ADLS --> Azure SQL DB with Modified time: In this case, Source(ADLS, wildcard(*.csv)), sink(SQL DB, Auto create
Table), Source, Filter by Last Modified Start Time, End Times dynamic content as below,

Start Time (@subtractFromTime(utcnow(),1,'Day'), End Time(@utcnow()), Sink, Table(Edit) --> dbo.nn-table, remain
debug pipeline same

Copy Each File to New Table: ADF --> pipeline --> parameters FileNames_pip, Array, ["file1.csv","file2.csv","file3.csv"]

ForEach, Sequential, Items, dynamic content(select FileNames_pip parameter), Inside ForEach activities drag copy activity
Source(ADLS), Inside ADLS dataset, parameters(Filename, string), connection(Filepath, dynamic content(select Filename
parameter), Next at pipeline level source FileName(select ForEach1 current item), sink(SQL DB, Auto Create Table),

parameters(tablename,string), Table Edit(dbo.dynamic content(select tablename parameter), Next at pipeline level sink Dataset properties tablename(dynamic content, @replace(item(),'.csv',' ')) --> debug pipeline

Set Variable:

ADF --> Author --> pipeline --> variable [(x,string), (y,string), (currentdate, string), (subtime, string)], Drag Set Variable --> [Variables(select x, value(Hello)), 2nd Set variable[Variables(select y, value(1000)), 3rd Set variable [Variables(select currentdate, value(dynamic content(utcnow))), 4th Set Variable [Variables(select subtime, value (dynamic content(@subtractFromTime(utnow(),5,'Month')))] --> debug

GetMetadata:

ADF --> Author --> pipeline --> drag GetMetadata, Settings, ADLS dataset, Field List(child items) --> debug

ForEach:

ADF --> Author --> pipeline --> parameter(New --> Countries, Array, ["India","US","UK"]), Variable(CountryName,string) drag ForEach, Settings --> sequential, Items(dynamic content, select Countries parameter) --> OK Inside ForEach Activities --> drag Set Variable --> select CountryName variable, dynamic content(ForEach1 current item), ok Next debug pipeline

Append Variable:

ADF --> Author --> pipeline --> parameter(New --> Countries, Array, ["India","US","UK"]), Variable[(CountryName,Array), (CountriesOutput, Array)], drag ForEach, Sequential, Items(dynamic content, select Countries parameter) --> OK Inside ForEach Activities --> drag Append Variable --> select CountriesOutput variable, dynamic content(ForEach1 current item), ok --> Next drag Set Variable beside ForEach & connect both --> Inside Set Variable [select CountryName, value(dynamic content, select CountriesOutput variable) --> run debug pipeline

Delete:

ADF --> Author --> pipeline --> drag delete --> source (ADLS, specific csv file or wildcard(*.csv)) --> debug pipeline

Lookup:

ADF --> Author --> pipeline --> drag Lookup --> Settings (ADLS dataset, check first row only) --> debug [Variables(select x, string)] --> Drag Set Variable --> select x, dynamic [content\(@activity\('Lookup1'\).output.firstRow.city\)](#) Debug pipeline

Lookup with Full Dataset: same as above, uncheck First Row only option, Next here Set Variable not suited, so we use ForEach activity combine with Lookup

ForEach --> settings(dynamic content, @activity('Lookup1').output.value), Inside Activities --> drag Set Variable, Inside Set Variable --> Variables (select x, dynamic [content\(@item\(\).id\)](#)) --> debug pipeline

Stored Procedure:

keep ready SQL DB, Table and Stored Procedure with parameterize

ADF --> Author --> pipeline --> drag Stored Procedure, create Linked service, select above Stored Procedure, click Import, next give value against parameter Ex: City, string, value(LONDON) --> debug pipeline

Filter:

ADF --> Author --> pipeline --> Parameters(Numbers, Array, [1,2,3,4,5,6]), drag Filter Settings[Items(dynamic content, click Numbers parameter), Condition(dynamic content, @greaterOrEquals(item(),5))] Condition2: @less(item(),5), Condition3: @not(equals(item(),5)) Condition4: @and(greaterOrEquals(item(),2),lessOrEquals(item(),5)) Parameters(Countries, Array, ["India","US","UK"]) Settings[Items(dynamic content, click Countries parameter), Condition(dynamic content, @equals(item(),'India'))] Condition2: @contains(item(),'a'), Condition3: @startswith(item(),'U')...etc

Lookup + Filter: drag Lookup & configure, drag Filter, Settings[Items(dynamic, @activity("Lookup1").output.value, Condition: @greaterOrEquals(int(item().salary),30000)

Lookup + Filter+ForEach(Stored Procedure): do same as above, next drag [ForEach\(@activity\('Filter1'\).output.value\)](#) Inside activities drag Stored Procedure, create Linked service, select above Stored Procedure, click Import next give dynamic values against parameters Ex: city, string, [value\(@item\(\).city\)](#), Name, string, [value\(@item\(\).Name\)](#), id, int, [value\(@int\(item\(\).id\)\)](#), Salary, int, [value\(@int\(item\(\).Salary\)\)](#) --> debug pipeline Note: keep ready SQL DB, Table and Stored Procedure with parameterize

If Activity:

ADF --> Author --> pipeline -->

Dataflow Transformations:

Select:

ADF --> Author --> Dataflow --> Source(Dataset ADLS, csvfile) change datatypes in projection, click on +, "Select" Transform, Delete unwanted columns, rename, reorder can do graphically, click on +, Sink(Dataset(SQL), output) Create New Table(dbo.nn-table) --> ADF --> Author --> pipeline --> drag the above dataflow, publish, Trigger

Sort:

Source(ADLS, csv file) + Select + Sort + Sink(SQL) same as above

Sort Conditions [select needed columns, give Ascending/Descending], Optimize(Single Partition)

Filter:

Source(ADLS, csv file) + Filter + Sink(SQL) same as above

Filter on (Expression builder, Country=="India", Country!="India", SalesAmount>=3000, SalesAmount>=2000 && SalesAmount<=5000, Country == "India" || Country == "Germany", startswith(ProductName,"Ra")

Case2:

Source(ADLS, csv file) + Filter(Country=="India") + SinkIndia(SQL, dbo.nn-tableIndia) same as above

Source(+), New Branch, (+), Filter(Country=="Germany"), (+), SinkGermany(SQL, dbo.nn-tableGermany)

ADF --> Author --> pipeline --> drag the above dataflow, publish, Trigger

Conditional Split:

Source(ADLS, csv file) + Conditional Split + Sink

Output Stream(SplitSales), Incoming Stream(above source name), Split Condition[G800, SalesAmount>800, L500 default]

2 Sinks [Sink800(ADLS, container-G800), Sink500(ADLS, container-L500)] --> debug pipeline

Source(+)New Branch + Conditional Split [G2012, year(OrderDate)>=2012, L2012 default]

2 Sinks [SinkGE2012(ADLS, container-GE2012), SinkL2012(ADLS, container-L2012)] --> debug pipeline

Casting:

Source(ADLS, csv file) + Cast[select columns. Change data types] + Sink(ADLS, output)

Uncheck Assert type check option [This option visualize the data type errors]

Surrogate Key:

Source(ADLS, csv file) + Surrogate Key + Sink(ADLS, output) [It is auto incremental id column]

Incoming stream(source name), key column(id), start value(1), step value(1) --> debug pipeline

Aggregate:

Source(ADLS, csv file) + Cast + Aggregate + Sink(ADLS, output)

GroupBy(Country), Aggregate[TotalSales, sum(SalesAmount), AvgSales, avg(SalesAmount)]

Sum, Avg, Max, Min, Count...etc can be done by using this Transformation

Case2:

GroupBy[year(OrderDate) name as SalesYear, toString(OrderDate, 'MMM') name as MonthName]

Aggregate[TotalSales, sum(SalesAmount), AvgSales, avg(SalesAmount)], Optimize(single Partition)

Pivot:

Source(ADLS, csv file) + Cast + Pivot + Sink(ADLS, output)

Pivot --> Group by (year(OrderDate) Name as SalesYear, EnglishProductName Name as ProductName), Pivot key (Country), Pivot Columns(open expression builder, Name(TotalSales), Expression(sum(salesamount)), Optimize(Single Partition) --> Next Sink, Next debug pipeline by drag this dataflow into canvas

Note: In pivot key value, if we kept blank it splits all countries into separate columns, if we give specifically India, US, it gives India & US values only

Un-Pivot:

Source(ADLS, above pivoted csv file) + Un-Pivot + Cast + Sink(ADLS, output)

Unpivot --> Ungroup by(EnglishProductName), Unpivot key[Unpivot column Name(Country), string]

Unpivoted Columns[Columns(SalesAmount)], Optimize(single Partition)

Cast(SalesAmount, double) --> Next Sink, Next debug pipeline

Rank:

Source(ADLS, csv file) + Rank + Sink(ADLS, output) [Optimize(single Partition)]

Options(tick Dense), Rank Column(Ranks), Sort conditions(SalesAmount, Descending), Next Sink, Next debug pipeline

Window Transformation:

Case1:

Source(ADLS, csv file) + Select + Window + Sink(ADLS, output) [Optimize(single Partition)]

Select --> keep needed 3 columns (OrderDate, salesamount, Country) only, remain delete

Window --> Over(Country), Sort(OrderDate, Ascending), Window Columns[TotalSales, Expression(sum(salesamount)), TotalCount, Expression(count(salesamount))], Next Sink, debug pipeline

Case2:

Source(ADLS, csv file) + Select + Derived Column + Aggregate + Window + Select + Sink(ADLS, output)

[Optimize(single Partition) for all]

Select --> keep needed 2 columns (OrderDate, salesamount) only, remain delete

Derived Column --> columns[(Month, month(OrderDate)), (Year, year(OrderDate))]

Aggregate --> Aggregates(TotalSales, sum(salesamount)), Group by[toString(OrderDate, 'MMM-yyyy'), Name as Month]

Group by[(Year, Name as Year), (Month, Name as Month)]

Window --> [Sort(Year, Month, Ascending both), Window Columns(MovingSum, sum(TotalSales), MovingAvg, round(avg(TotalSales),2))]

Range by[untick Unbounded, Range (drag Behind, Current row), Start offset(2), End offset(0)]

Select --> delete Month, Year columns, keep remain same, Next Sink, Next debug pipeline

Union Transformation:

3 Sources(ADLS, 3 csv files) + Union + Sink(ADLS, output)

Source1 + Union --> Incoming stream(source1), union by(name/position), union with(source2, source3) --> debug pipeline by drag Dataflow

Lookup Transformation:

3 Sources(ADLS, 3 csv files) + Lookup1 + Lookup2 + Select + Sink(ADLS, output)

Source1 + Lookup1 --> Primary stream(source1 name), Lookup stream(source2 name), Lookup condition

[Left source1 column(ProductKey) == Right source2 column(ProductKey)] [data & data types of both should match]

Lookup1 + Lookup2 --> Primary stream(Lookup1), Lookup stream(source3 name), Lookup condition

[Left Lookup1 column(CustomerKey) == Right source3 column(CustomerKey)]

Select --> remove duplicate columns, keep remain same, Next sink, Next debug, [Optimize(single Partition) for all]

Exists Transformation:

2 Sources(ADLS, 2 csv files) + Exists + Sink(ADLS, output) [Optimize(single Partition)]

Source1 + Exists --> Left stream(source1), Right stream(source2), select Exists option, condition[ProductKey == ProductKey]

If we choose Doesn't Exists, it will give unmatched records/rows of Left stream(source1) only

Join Transformation:

3 Sources(ADLS, 3 csv files) + Join1 + Join2 + Select + Sink(ADLS, output) [Optimize(single Partition)]

Source1 + Join1 --> Left stream(source1), Right stream(source2), choose type[Full/Inner/left outer/right outer/custom] condition[ProductKey == ProductKey]

Join1 + Join2 --> Left stream(Join1), Right stream(source3), choose type[Full/Inner/left outer/right outer/custom] condition[CustomerKey == CustomerKey], Next Select(remove duplicate columns), Sink, debug pipeline

Flatten Transformation:

Case1:

Source(ADLS, Json file) + Flatten + Sink(ADLS, output)

Source --> Json Settings(Array of documents)

Flatten--> Income stream(source name), Unroll by(Items[]), Next sink, next debug pipeline by drag dataflow

Case2:

Source(ADLS, Nested Json file) + Flatten + Sink(ADLS, output)

Source --> Json Settings(Array of documents)

Flatten--> Income stream(source name), Unroll by(Phone[]), click reset, add other needed columns, Next sink, next debug pipeline by drag dataflow

Azure Devops:

Its fully Managed cloud-based service that automates the process of building, testing, deploying code. With powerful features like Yaml-based config, automated testing and flexible deployment options, Devops practices are to deliver applications quickly and with high quality. By using Yaml pipelines development teams can automate build, test and deployment processes efficiently.

Yaml pipeline is defined by several key sections that represent the stages, jobs and tasks in CI/CD process. Each section helps manage different steps like building, testing and deploying the application. Main components of pipeline are as follows:

Trigger, Pool, Stages and Jobs, Variables, Artifacts, deployment

Benefits/Features of Azure Devops Yaml pipelines: Version Control, Flexibility, Re-usability, Portability

To Create Release pipeline, by pre-built Templates (Azure App Service deployment, Kubernetes deployment) or start with an empty job. Release pipeline is divided into stages that represent different env [dev, staging, prod]

Link Artifacts: Choose latest version of Artifact (output of CI build pipeline) & configure pipeline

Triggers: Enable Continuous Deployment Trigger to automatically create a release when a new build is available.

Variables: By using variables, we can define different values for each stage (dev API key, prod API key).

Approval Gates: With this, we can control release pipeline by manual approval before deploying to production

Benefits of Azure Devops: Increased Development Speed, Early Bug Detection, Improved Code Quality, Consistency in Deployment, Scalability

Azure Databricks is an Apache Spark-based analytics platform designed for big data processing with interactive workspace notebooks to collaborate on large-scale data transformations.